

SilverLine: Preventing Data Leaks from Compromised Web Applications

Yogesh Mundada
School of Computer Science
Georgia Tech
yhm@cc.gatech.edu

Anirudh Ramachandran
Nouvou
avr@nouvou.com

Nick Feamster
School of Computer Science
Georgia Tech
feamster@cc.gatech.edu

ABSTRACT

Web applications can have vulnerabilities that result in server-side data leaks. Securing sensitive data from Web applications while ensuring reasonable performance and without requiring developers to rewrite entire applications is challenging. We present SilverLine, which prevents bulk data leaks caused due to code injection in Web applications as well as compromised user-level processes on the application server. SilverLine uses login information to associate a user with each Web session; it then taints each file and database record and applies information-flow tracking to the data associated with each session to ensure that application data is released only to sessions of authorized users. SilverLine focuses on isolating data between user sessions and is thus most suitable to applications that involve single user sessions (*e.g.*, banking, e-commerce). We have implemented SilverLine on Linux; our implementation demonstrates that SilverLine can protect a PHP-based Web application from many of the most common server-side Web application attacks by modifying only about 60 lines of code from the original application. Our evaluation shows that SilverLine incurs a performance overhead of about 20–30% over unmodified applications.

1. INTRODUCTION

Web applications offer users a range of services, including email, social networking, online banking, stock trading, and subscription-based audio and video streaming services. In certain cases, corporate employees may also use Web applications to access confidential information. Developers of Web applications may use a variety of programming languages, Web servers, load balancers, caches, Web application frameworks, in-memory key-value stores, and databases—each of which may have vulnerabilities. A vulnerability in any of these components leaves the entire Web application susceptible to attacks. As a result, many recent data leaks have exploited vulnerabilities in Web applications to access private files and databases. According to a recent report, about 90% of data leaks occur at servers, and 95% of those leaks are caused by external attacks [4]. Web application vulnerabilities have led to attacks in many industries from finance [14] to large enterprises [2] to healthcare [13]. Such compromises can be devastating. For example, Sony Corporation's Web properties were compromised us-

ing SQL injection attacks during March and April 2011; attackers stole and publicized personal and credit card information [2]. The cleanup cost from this incident cost Sony \$171 million, with as much as \$20 billion of indirect costs [9]. Similarly, Citibank suffered a breach of 360,000 user accounts due to a vulnerability in their online banking application, which allowed one user to access another user's account by merely modifying an account number field in the URL [14].

Because most Web applications have complete access to databases, once the Web application is compromised, an attacker can typically access any information in the database. The Open Web Application Security Project's top-10 Web vulnerabilities [25] include six server-side vulnerabilities that can result in data leaks. For example, a SQL injection attack occurs due to a Web application executing a SQL query without properly sanitizing user input. Insecure direct object reference and local file inclusion also result from poor access control checks in the Web application.

Unfortunately, building security or better access control into any given Web application framework offers limited benefits, since Web development frameworks remain in constant flux. State-of-the-art approaches scan content to block certain types of traffic: to block incoming attack traffic, organizations use Web application firewalls that scan input to a Web application to determine potential attack traffic (*e.g.*, user input that matches a regular expression for SQL injection), or data loss prevention (DLP) devices that scan outgoing data from the Web application for sensitive data patterns. Web application firewalls can only block known attack patterns so they cannot defend against new vulnerabilities or zero-day attacks. DLP systems usually have one-size-fits-all policies (*e.g.*, do not let any credit card information leave unencrypted), and are not useful for protecting Web applications that legitimately need to send and receive sensitive information.

Existing defenses check the Web application itself for vulnerabilities or perform penetration testing so as to search for flaws in the same way that attackers do. Unfortunately, these approaches cannot defend against all zero-day attacks, bugs, or vulnerabilities in future versions of the Web application. Once compromised, Web applications are vulnerable to data leaks; existing defenses such as network firewalls offer little protection for the sensitive data itself. Another possible approach to protecting against data leaks is to isolate each Web session in a virtual machine, as is done in CLAMP [27]. CLAMP provides strong guarantees against data leaks, but cannot scale to large number of users, since it requires the server to spawn a virtual machine for each user.

Rather than focusing on creating perfectly secure Web applications or blocking all attack traffic, we posit that application compromise is inevitable and, hence, that the underlying system should ensure that data remains secure, even if the application itself is

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. To copy otherwise, to republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee.

ACISAC '13 Dec. 9-13, 2013, New Orleans, Louisiana USA
Copyright 2013 ACM 978-1-4503-2015-3/13/12 ...\$15.00.
<http://dx.doi.org/10.1145/2523649.2523663>

compromised. Towards this goal, we present SilverLine, a system that tracks the flow of each user’s data between the components of a Web application. SilverLine trusts only a small portion of user-level code on the Web server that authenticates that application’s users; the small size of this module could permit formal correctness verification. SilverLine places this module into a trusted realm. Thus SilverLine entrusts the attestation provided by authentication module for a specific user’s session and permits outgoing data on client Web sessions only if the user is authorized to retrieve the sensitive data. Even if an attacker gains access to sensitive data by compromising a Web application or other user-level processes, SilverLine prevents that data from being leaked, because the attacker’s Web session cannot prove that it is authorized to access the data.

SilverLine associates a *taint* with each piece of data that the Web application accesses, such as a database record or a file. To protect data independently of the applications that access it, and to track information flow, SilverLine instruments the operating system kernel of the Web application server with a trusted module that associates a taint with each process and file. The SilverLine kernel module propagates these taints as files and data objects are accessed or modified; the kernel propagates these taints regardless of whether the data is transferred within a single host or to another host on the network. When a Web application reads a user’s data, SilverLine propagates the data’s taints through each Web application component, and associates them with network packets that the Web application generates. SilverLine’s declassifier module then checks the taints on the outbound flow with the set of taints that the user is authorized to see. If they match, the traffic flow is allowed; otherwise, the flow is blocked. Although we have implemented SilverLine’s declassifier as a standalone control application, it decides whether to allow flows based on taints that the flow carries and could thus be implemented as an application as a *Software Defined Networking (SDN)* control application in an existing OpenFlow [23] controller (e.g., Pox [30], Floodlight [11]).

SilverLine separates Web application security from the security of its data, ensuring that a user’s data is secure even if the Web applications that access the data are compromised. Because SilverLine performs information-flow tracking at the operating-system level, it can defend against attacks on any application or service that touches the data, not just attacks against the Web application. Because it does not require virtual machines to isolate data from different users, it reduces throughput by only 20–30% compared to a native system without any data leak prevention, which is significantly less performance overhead than existing approaches (the state of the art, CLAMP, incurs a 42% reduction in throughput).

This paper presents the following new contributions: First, we present SilverLine, a system that prevents data leaks in vulnerable Web applications that is independent of any particular Web application and requires only minimal changes to existing Web applications. As part of designing and implementing SilverLine, we develop a new mechanism for associating taints with Web application users. We also develop a framework that makes it easy for administrators to configure SilverLine policies. To show that SilverLine is practical and scalable, we port a widely used real-life application that is used in more than 12,700 online stores, OSCommerce [24], to run on SilverLine. We show that SilverLine imposes less overhead than existing data leak prevention systems and scales to many more users and requests than these existing systems.

2. DESIGN GOALS

SilverLine protects multi-user Web applications where a single user logs in over a session. Figure 1 shows a typical Web application deployment scenario and the set of threats that we address in

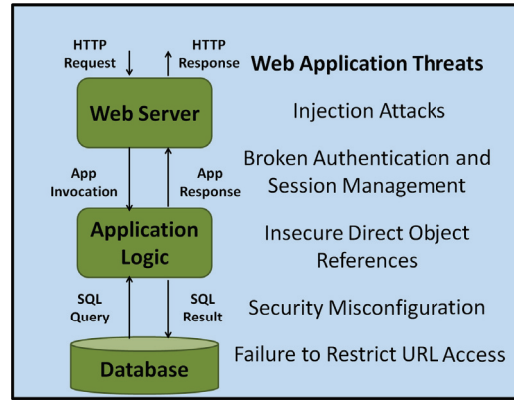


Figure 1: Web server stack and threats that SilverLine defends.

this paper. We aim to defend against data leaks in the case where an attacker can compromise the Web server, user libraries, or the Web application code itself. An administrator for the Web application must identify the sensitive data and apply the appropriate initial set of taints; we believe that this approach is reasonable because the sensitive data accessible through Web applications is easy to identify and protect in advance. We aim to achieve the following goals:

Security: Decouple data protection from the application. SilverLine prevents data leaks, but it does not protect the Web application from exploits. By identifying private information when the database schema is designed, SilverLine can prevent data leaks independent of specific queries or application compromises. SilverLine may still be vulnerable to certain types of data leaks, such as covert channels, but it can defend against any data leak attack that results from the application having complete access to the back-end database (a common scenario for Web applications).

Performance: Minimize overhead. We aim to keep the performance overhead of deploying SilverLine as low as possible, by reducing the overhead associated with tracking data in Web sessions. In contrast to existing systems such as CLAMP, SilverLine does not require spawning a new virtual machine for each new Web session, which improves both scalability and performance.

Deployment: Minimize changes to existing applications. Due to the large deployed code base for existing applications, our goal was to allow SilverLine to be deployed for existing Web applications, without requiring significant rewrites of application code. SilverLine only requires applications to have a secure login module, that associates Web sessions with particular users. Other than the introduction of this secure login module, SilverLine does not require any other changes.

“Non goals” and vulnerabilities. SilverLine also has several “non-goals”. SilverLine only defends against attacks on user-space applications (e.g., the Web application), not on the operating system that hosts the application; kernel code injection could disable SilverLine’s information-flow tracking. SilverLine does not protect against malicious software on the database server that might directly exfiltrate sensitive data without passing through SilverLine’s data firewall¹. SilverLine also does not consider threats posed by insiders or others who have physical access to machines that hold sensitive data. Because SilverLine protects data rather than iden-

¹This is a reasonable assumption: we expect that the database server is firewalled from being directly accessed through the public Internet. Furthermore, if the database server software was malicious, it could return sensitive database records even for non-sensitive queries.

tifying malicious input, it cannot defend against malicious attacks that modify data. Because SilverLine performs no additional access control on data flowing *into* the Web application, an attacker might still be able to make the Web application execute harmful commands (e.g., a “DROP TABLE *” query using SQL injection); we believe that these attacks are less serious than data leaks, because periodic database and transaction log backups make it possible to restore the database at a five-minute granularity [1].

3. DESIGN

We first present an overview of the SilverLine design and describe how the system integrates with a current three-tier Web architecture. We then explain how taints are managed and propagated, and how SilverLine applies information-flow control to prevent unauthorized data access.

3.1 Overview

Restricting information access requires knowledge of which rows a database query will select, which can be difficult in practice and also restrict the Web application developer. Many existing systems do not allow one user to query another user’s data [27]. They also do not allow queries from an anonymous context, thereby making aggregate queries difficult or impossible. In SilverLine, application code and database queries can access all data (thus making aggregate queries easier to perform), but exfiltration is restricted based on the user who attempts to access the data, which provides significantly more flexibility.

SilverLine has the following trusted components: the *database proxy*, which applies taints to retrieved data; the *user authentication* component; and a *network-wide information flow control system*, which propagates taints. SilverLine taints database records by augmenting a user’s database records with an additional “users_taint” table, and using a database proxy that sits behind the database server and the Web application to transparently apply these taints before data is returned to the Web application. We provide a brief overview of each module below.

Database proxy. Requests sent to or received from a sensitive database pass through a trusted *database proxy*, which can identify database requests for sensitive data, and appropriately taint the responses before they are sent back to the untrusted Web application. This proxy allows the Web application to function exactly as it does without SilverLine—it fetches sensitive data from the database, processes it, and returns it to the logged-in user’s session. This approach makes it easier to deploy SilverLine with existing Web applications.

Authentication module. A legitimate user of the Web application who accesses the landing page may be prompted to login using code served directly by the trusted *authentication module*. The authentication module uses a separate, trusted database to authenticate the user and sets policies for the *application* firewall; these policies specify the types of sensitive data that the user is allowed to see. The authentication module then forwards the user’s request and the token indicating successful authentication to the core Web application. As in similar systems, the authentication module must be trusted. *The authentication module is the only part of existing applications that needs to be rewritten.*

Information flow monitor. The Web application runs on a *network-wide information-flow control system*. SilverLine uses a kernel-level module to track the flow of information from tainted sensitive data to applications. Much like taint-tracking, it can propagate taints between files and processes, but can also propagate taints across a network—a key requirement for distributed Web

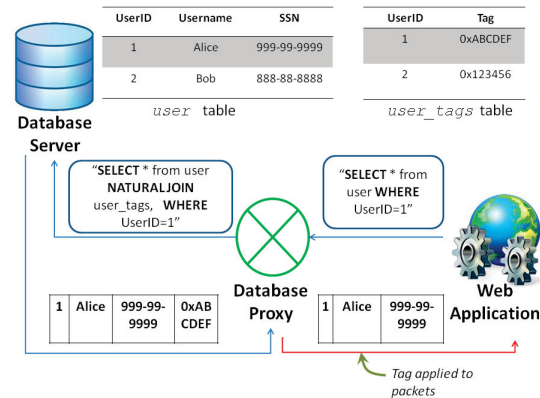


Figure 2: Rewriting retrieves taints associated with the result.

applications. After the Web application processes sensitive data, the responses it generates are forwarded through a simple programmable firewall component, the *declassifier*. The declassifier relies on the authentication module to compare any taints on outgoing network flows with the authorization associated with the client session: if the user is authorized to read data with a certain taint, the declassifier forwards the data; otherwise it drops it. Thus, even if the attacker compromises the Web application, the attacker’s session cannot read the data.

3.2 SilverLine, Step-by-Step

Step #1: Administrator tags sensitive data. SilverLine relies on the database administrator (DBA) to mark sensitive information in the database tables. It also requires the DBA to specify primary, candidate, and foreign keys, as well as referential integrity constraints between them. For example, a sensitive table may contain the primary key *User-ID* and foreign key *Transact-ID*. This foreign key might be used in another table, *Transaction*, as a primary key. SilverLine creates a unique taint for each user and stores taints for all users in a separate table *User-Taints* with *User-ID* as the primary key. Storing taints for data in a separate table prevents the need for drastic changes to application logic or the database proxy if the database schema changes, although it incurs some additional overhead when joins occur. All tables with private data might not have a *User-ID* as primary key. Querying one of these tables would thus require multiple joins between tables, first to find the *User-ID* and then to locate the associated taints, which would slow performance. To reduce performance overhead in these cases, SilverLine generates additional taint-storage tables for each foreign key. For example, Figure 3 shows a taint-storage table, *Transact-Taints*, corresponding to the *Transaction* table.

Step #2: The authentication module associates a user with a session. The application’s authentication module performs user logins by comparing usernames and passwords input by clients with the hashed password database of the Web application. New client requests to the Web application first encounter a login Web page hosted by the authentication module. After users enter their username and password, the login module accesses the *User-Authentication* table to authenticate the user. If the login succeeds, the module performs two actions. First, it creates a new Web cookie, returns the cookie to the client, sets the cookie in a *User-Sessions* table indexed by the user ID, and redirects the user to the main Web application. The Web application can now use the *User-Sessions* table to verify that the client is authenticated. Second, it creates a new record in a *Connection-Capabilities*

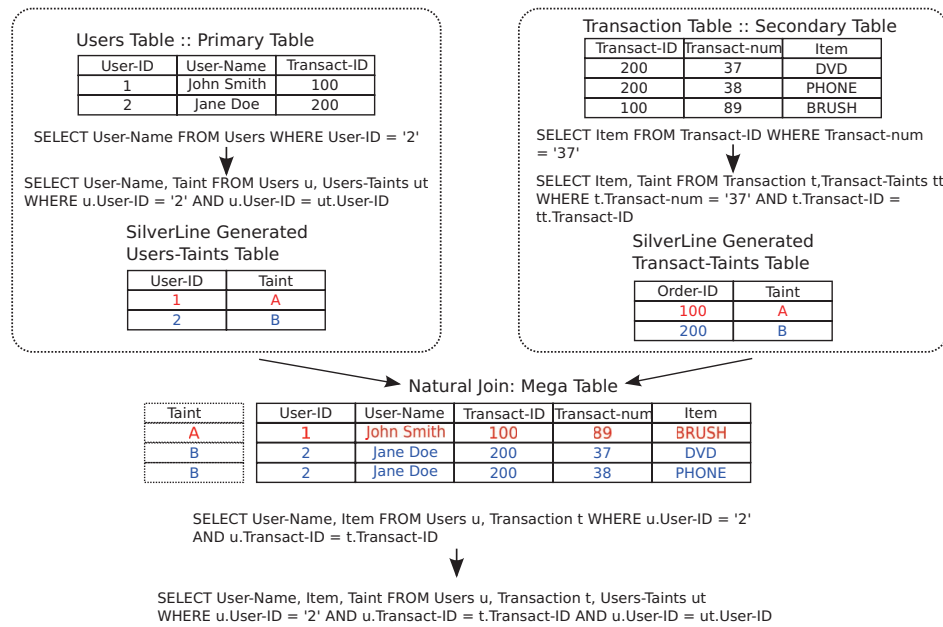


Figure 3: Example of how SilverLine stores taints and associates them with records belonging to each user.

table that specifies the set of sensitive taints data may carry over the user's connection; a connection is identified using the 5-tuple of source IP address, source port, destination IP address, destination port, and protocol. The *Connection-Capabilities* table is set by querying the *User-Taints* table. Web applications typically need to distinguish between the users who may have access to particular data, which allows SilverLine to track taints at a per-user, per-process level. These coarse-grained taints incur less performance overhead than other information-flow control systems.

The authentication module is the only component in existing applications that involves refactoring within an existing Web application. First, the module must be trusted and hardened against compromise; otherwise, an attacker could masquerade against any user to leak data. Second, we ensure that the authentication module is the only module that can access a user's password; otherwise, a compromised Web application could modify users' passwords to gain access to sensitive data. The *User-Authentication* table is also stored in the same database used by the Web application, but SilverLine relies on the trusted database proxy to block direct modifications to this table from the Web application.

SilverLine applies taints at the granularity of processes, which reflects a particular design choice in the tradeoff between performance and security. Tainting each thread independently would likely incur less performance overhead, but such tainting would have to take place in user space, where malicious processes could be more capable of interfering with or compromising the tainting process. Tainting at the process level incurs more overhead by comparison, but allows all tainting to take place in the (trusted) kernel.

Step #3: The database proxy rewrites user queries to retrieve taints associated with a result set. SilverLine's database proxy parses and rewrites queries or responses to inter-operate with the SilverLine back-end, as shown in Figure 3. Database proxies such as MySQL proxy [21] are becoming increasingly popular in the industry for database activity monitoring, query logging, and for detecting queries that may damage the database.

The *User-ID* field in *User-Taints* refers to the primary key of *Users*. When a SELECT query is issued to retrieve a user's

data, the database proxy parses the SQL to identify whether the query attempts to retrieve sensitive data (based on policies that the administrator specified in the schema). The proxy then modifies the query to execute a join with the *User-Taints* table; the associated taints are then applied to the result set. We found that about 85% of OSCommerce's queries fall into simple SELECT queries that can be rewritten in this fashion. Still, Web applications sometimes perform complex join queries that may be harder to rewrite.

Figure 3 shows how SilverLine stores taints and associates them with users for an example database schema involving users and their associated transactions. The *Users* table is a primary table that associates *User-ID* with *Transaction-ID* and can be directly joined with the *User-Taints* table to retrieve taints associated with each user. When a query is executed against *Users* the SilverLine database proxy finds taints associated with each user ID by performing a simple join between the *Users* and *User-Taints* tables.

We that of the 50 tables in the OSCommerce Web application, only 15 required taints; nine of those tables could be associated with *User-Taints* table since they shared same primary key. Out of the remaining six tables, five of them shared same primary key and hence a taint-storage table with that foreign key allowed to extract the taint. In the end, we needed to create three taint-storage tables in order to extract taints by doing just one join.

To retrieve taints for queries that perform joins, SilverLine first discovers all of the sensitive tables associated with that query and determines the taint-storage table with the correct primary key. The database proxy then modifies the original query to add the taint-storage table in the join, from which it selects the taint column.

Step #4: The database proxy associates taints with the response. When the Web application requests records for a user, the database proxy associates the user's taint with the outbound connection between the database server and the Web application, using a trusted, shared cache that trusted SilverLine components can all access. Because the Web application runs on an OS that includes the SilverLine kernel module, the host can pick up the taint on the incoming data and associate the taint with the Web application pro-

cess that reads the data.

Step #5: The server’s information flow control system tracks reads and writes associated with the tainted record. Once taints are associated with any resource (typically a file or a process), the kernel monitors any system calls that transfer information from the resource, and propagates the taints to the destination of the information flow. If a process reads a tainted file, the process acquires the taints of the file. If a tainted process writes to a file, the file acquires the taints of the process. Finally, if a tainted process sends data over the network to another process, the receiving process acquires the taints of the sending process.

To propagate taints between processes on a single machine, the kernel intercepts each system call and checks whether the sender resource carries any taints. If it does, it adds all of the sender’s taints to the receiving resource’s set of taints. To propagate taints across the network, the sender’s taint-tracking module creates an entry in the `Connection-Taints` table that maps the 5-tuple for the flow with the current set of taints of the sending process. This out-of-band taint transfer keeps implementation simple and avoids intrusive changes to the underlying system. When a host receives the first packet of a flow, it queries the `Connection-Taints` table and applies the set of taints to the receiving process. A sender can use the Type of Service (TOS) field in the IPv4 header of the outgoing packet to indicate additional taints.

Step #6: The declassifier determines whether a user can access data.. The *declassifier* determines whether a particular flow should be permitted to go to a client Web session. When the server sends the response back to the client for the first time, the declassifier verifies that the connection has sufficient privileges to carry taints associated with it by querying `Connection-Capabilities`. Once the declassifier decides that the client should have access to the data, it will not check for violations until the sending process acquires new taints (as indicated by TOS field on incoming packets). If the data in the flow should not be allowed, the declassifier terminates the connection and sends a signal to the server to kill the associated processes. We have implemented the declassifier as a standalone application that is on the path between the client and server, but in the future it could be implemented as an off-path network control application using an SDN controller such as Floodlight [11] or Pox [30]. Flow table entries permitting flows could be installed in the switches using micro-flow entries that match on the TOS field and other flow tuples; a change in the TOS bits would result in a “miss” in the switch flow table, causing redirection of the traffic flow to the declassifier, which could subsequently re-evaluate whether the corresponding traffic should continue to be forwarded.

4. IMPLEMENTATION

This section describes the implementation of SilverLine components; our integration of SilverLine with OSCommerce, a popular Web commerce application; and the process by which an administrator configures SilverLine.

4.1 SilverLine Components

We describe the implementation of SilverLine’s *trusted components*: the authentication module, the database proxy, and the information flow control framework.

4.1.1 Authentication module

The login process of the Web application has two components. The request handler, executes on the Web server, as before. However, it only handles the incoming login requests. The actual authentication is performed by another process, which runs on a se-

Syscall Type	Example syscalls
Inter-process Communication	send(2), shmatt(2), msgsnd(2), kill(2)
File/device operations	read(2), unlink(2), mknod(2)
Process creation	fork(2), exeve(2), clone(2)
Memory operations	mmap(2), mprotect(2)
Kernel configuration	sysctl(2), init_module

Table 1: Types of system calls that are monitored by the labeler, with examples of each system call type.

cure server. When the request handler receives a login request from a user, it invokes the RPC of the authentication process with username, password, and the five-tuple of the login request. This process then checks whether the username and password are correct. If it does, it retrieves the taints associated with that user and puts them in the `Connection-Capabilities` with the associated five-tuple and returns a success code to request handler. The request handler then generates a session for that user, creates a cookie, and returns the cookie to the user. **We implemented these changes in less than 60 lines in the OSCommerce source code.**

4.1.2 Database proxy

We wrote a 350-line Lua script that acts as a MySQL database proxy, which intercepts queries coming from web application and decides if any of them will potentially retrieve sensitive information. If the proxy identifies a query that accesses sensitive information, the proxy then discovers the taint associated with that information and applies it to the result of the query.

Taint Storage. As mentioned in the design section, we create new taint-storage tables for each primary key for tables that contain sensitive data. If a taint-storage table shares its primary key with a bunch of database tables then it can be used to retrieve taints associated with any data accessed from those tables. Adding a new primary key to any of these tables requires adding a taint to the corresponding taint storage table. The database proxy monitors inserts in these tables and creates a new query that inserts a taint along with the primary key from the original insert.

Taint Retrieval. When the database proxy receives a database query, the proxy script runs a query parser. This parser analyzes the query and returns the query that will also retrieve the associated taints for that data, which is a join between the original table and the taint-storage table. It also returns a regular expression for the original query (to match similar future queries with different parameters) and a regular expression for the taint-retrieval query. The database proxy stores both of these queries in a global table for future queries. Any future queries that match the regular expression need not be passed to query parser, since the associated taints for those queries have already been retrieved.

4.1.3 Information flow control

Host-level information flow. We implemented host-level information flow control as a kernel module on Linux, using the Linux Security Modules framework [36] in the Linux kernel. LSM is a framework within the Linux kernel that allows various security models to be added-on without changing core kernel code. LSM provides hooks within system call handlers that can be implemented by a security module; thus, a third-party module can implement mandatory access control for a system call (e.g., `read(2)`) without changing the core implementation of the system call handler (e.g., `sys_read`). Using LSM hooks, the labeler intercepts all system calls that transfer information between host resources. We have implemented hooks to track information flow for the system calls in Table 1.

We expect that the module could be bundled with the OS distribution for the machines running the Web application or installed when the system is in a known clean state. On boot, the module is loaded shortly after `init` during the boot sequence. The hooks for the system calls are 1,500 lines of C code, and the logic for the module is another 6,500 lines, so the total module implementation is about approximately 8,000 lines of code.

Information flow across components. SilverLine use trusted databases to communicate information from different components—from the login module to the Web application, from the kernel module to the declassifier, etc. For performance reasons, and to disassociate ourselves from the Web application’s own database, we use a single centralized high-performance key-value store for storing the `Connection-Capabilities`, `User-Sessions`, `Taint-Policy`.

We choose the Redis [31] key-value store for our implementation. Redis supports about 110,000 `SETS` per second, about 81,000 `GETS` per second [32], outperforming many relational databases (e.g., MySQL) and key-value stores (e.g., memcached). Redis supports only string keys, but values can be of any type. Redis store maintains three tables. The first table contains tag capability information. The second table contains the user information. The third table maintains which tags are *allowed* to be associated with the data leaving the network on each TCP/IP connection. These three tables are *protected* and can only be updated by trusted components such as database proxy. This table is filled by trusted authentication module after verifying the user’s credentials. The last table contain information about which tags are *acquired* by each connection. This table is updated by both database proxy and the Web application server. To protect untrusted user space code, on the Web application server, from updating entries in any Redis table, the end-host component of SilverLine does not allow any user space code to connect to the Redis server.

4.2 Configuring SilverLine

All SilverLine configuration occurs at setup time. A key feature is that the administrator who configures SilverLine does not have to understand the logic for the applications that run in the environment. The administrator does need to identify sensitive data for each user; in particular, the administrator must identify (1) the primary keys for tables, (2) groupings of tables with the same primary keys, (3) relationships between the foreign keys across tables and (4) the table from each group that shares a specific primary key for `INSERT` statements that would result in a new primary key for that group.

When porting OSCommerce to SilverLine and inspecting other Web applications, we noticed that administrators typically already intuitively identify many of these parameters. Once the administrator specifies these four parameters, SilverLine creates taint-storage tables for each group of tables and automatically maintains them whenever a row is inserted or deleted in that group.

5. PERFORMANCE EVALUATION

We now explore SilverLine’s performance and compare it to existing state-of-the-art systems with similar goals, such as CLAMP. SilverLine’s main performance overhead results from retrieving taints from the database server and propagating those taints to processes corresponding to that user on the Web server. This tracking involves executing the query, associating the taints with a connection in the central monitoring database, and performing taint tracking within a host using system call hooks. Table 2 summarizes the main results from the evaluation.

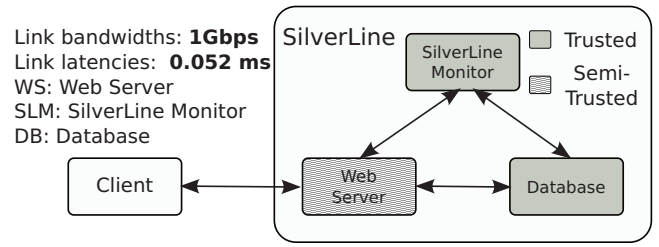


Figure 4: Evaluation setup.

For small files, the latency overhead constitutes about 7%; for large files, it is about 1%.	§ 5.2
SilverLine can support on the order of 1,000 simultaneous users with realistic response-time bounds.	§ 5.3
SilverLine’s overhead on the throughput of login requests is about 20%; the overhead on response time is about 29%.	§ 5.3

Table 2: Summary of performance evaluation.

5.1 Experiment Setup

To measure SilverLine’s performance overhead, we ran various experiments with and without SilverLine. Without SilverLine, we ran the experiments on the native LAMP stack. With SilverLine, we enabled the authentication module, taint tracking, and a MySQL database proxy to fetch and apply taints.

Our testbed consisted of four Emulab nodes connected in a setup as shown in Figure 4. Each node had 64-bit 2.40 GHz Intel Quad Core Xeon E5530 with 12 GB RAM. Each link is point-to-point with 1 Gbps capacity and roughly 0.05 ms latency. We ran evaluation scripts on the client node. The server runs the OSCommerce web application with the taint-tracking kernel module. The database node runs a MySQL server with the SilverLine MySQL database proxy. The “SilverLine Monitor” node hosts the trusted databases that perform information-flow tracking across nodes. The monitor and database nodes contain only modules that are completely within the trusted domain. The Web server node, which runs the Web application that also has the (modified) authentication module, has both trusted components (i.e., the authentication module) and untrusted components (i.e., the Web application itself).

We used the Apache Web server on the server with a prefork `mpm` module to allow only one client connection per child process. The server spawns a new child process per client connection to avoid taint diffusion across different client sessions, since tainting is performed on a per-process granularity. To prevent multiple requests from the same client being served by separate child processes, we also enabled the `Keep-Alive` option. Apache can spawn about 32 child processes per second. Under higher request loads, the default Apache server was unable to fork enough children quickly enough, so we modified the spawning algorithm of the Apache `mpm` module. (It is also possible to use other Web servers that can fork more child processes to satisfy this constraint.) We configured frequently updated tables (e.g., sessions, shopping cart contents) using MySQL’s `innodb` storage engine to take advantage of row-level locking.

5.2 Latency

The ultimate measure of Web application performance is perceived user experience, but it is extremely difficult to measure this

metric. As a proxy for this metric, we measure the latency experienced by the end user due to SilverLine. In Section 5.2.1, we perform macrobenchmarks to measure overall latency increases incurred when serving both static files as well as requests that read or modify the back-end databases. In Section 5.2.2, we perform microbenchmarks to understand the contribution of each SilverLine module to latency.

5.2.1 Macrobenchmarks

We measure performance of various operations when the web server is not loaded with multiple requests. We measure all the times from the client machine. A single type of operation such as “Login” may consist of more than one web request; therefore, we start measuring the time when the first HTTP request is issued and stop measuring it when all the requests in that operation have responded. For each operation, we perform 50 trials, and report average and standard deviation of latency. Figure 5 shows these results. In all of our experiments below, the link latency between the client machine and the Web server is 0.05 ms, which is much smaller than it would be in reality on the wide-area Internet. We chose a small link latency to allow us to provide an upper bound on the latency overhead that SilverLine induces. In realistic settings, the percentage overhead that SilverLine induces will be an even smaller fraction of overall latency.

We replicated the same test cases as CLAMP to allow us to compare the two approaches. In all test cases, however, *CLAMP omitted the time to fork a new virtual machine* and assumed that virtual machines are pre-forked, which makes their performance numbers appear significantly better than they might be in practice, when new sessions would require forking new virtual machines. In contrast, *SilverLine’s end-to-end performance measurements also include time taken to spawn a new process that serves that client*. Additionally, CLAMP requires each session to be served over SSL to identify each session. We do not have such a constraint with our prototype and can serve requests with or without SSL transport. Thus, we chose to measure performance without SSL.

In the first set of measurements, we fetched small (8 KB) and large (3 MB) static files that did not involve database operations. To measure the performance overhead of SilverLine, we added single taint to each file. This static file transfer measures the overhead of taint tracking on a single host. *For small files, the latency overhead constitutes about 7%; for large files, it is about 1%.* File size does not affect the latency overhead associated with taint tracking, so for large files, that cost is amortized over the total file fetch time. Typically, a Web page is hundreds of kilobytes or more, thus making the percentage overhead of this fixed cost smaller. In contrast, CLAMP’s latency overhead increases as file sizes increase.

To test the login overhead, we measured the time to redirect to the monitor for secure login, retrieve taint from the database, and apply that taint to the user session. Overall, this function adds about 13 ms (30% in the worst case).

5.2.2 Microbenchmarks

To analyze how much each SilverLine component contributes to the latency overhead, we simulated a normal user operation and measured the time taken by each component. In this case, a user logs in, checks her cart that already has a few items added, visits a few more product pages and then logs out of the system. In terms of SilverLine components, the overhead involves redirecting to the monitor for secure login, retrieving the user taint, applying that taint to the user session and finally making a decision whether to let the information to be passed to the user browser. We found that the login process contributes about 8% of the overall latency

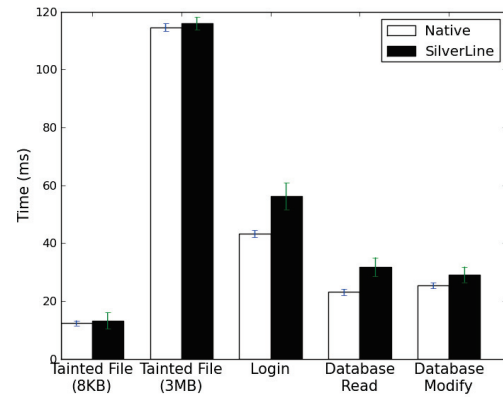


Figure 5: Macrobenchmark Latency. Comparison between native LAMP installation and SilverLine setup to complete various requests. Smaller is better.

overhead. The database proxy contributes 61% of the total latency overhead. Propagating taints to the Web server via monitor contributes 21%. Finally, taint tracking within the Web server and de-classification checks contribute 10%.

Finally, we measured the overhead of performing database operations after the user has already logged in. This measures the overhead experienced due to addition of MySQL proxy server that retrieves and propagates taints. The operation involved about 18 SELECT queries introduces an additional 8 ms overhead; the database update includes 20 INSERT or UPDATE queries and introduces a latency overhead of 4 ms.

5.3 Scalability

Since SilverLine does not consume much extra memory per process, the upper bound on simultaneous user connections (without considering the response time) is equal to maximum number of server processes that Apache Web server can spawn. Often, the Web server has a hardcoded limit of 20,000 server processes in total. We are able to support on the order of 1,000 simultaneous users with realistic response-time bounds.

To determine whether SilverLine can be deployed without affecting the scalability of the Web application, we generated parallel requests at the Web application and measured the resulting throughput as the number of sessions increased. Because most of the standard benchmarks do not simulate an actual user’s navigation for a specific Web application, we created our own set of tests using a Python and Curl framework. These testing scripts spawn multiple threads and perform various user actions such as logging in, browsing, adding items to shopping cart, checking out, and logging out. In general, we found that the throughput involving concurrent login requests only is reduced by 20%, while tests simulating multiple realistic user sessions had 30% lower performance. Most of the throughput reduction comes from the SilverLine database proxy that rewrites and retrieves taints. We found that even with this performance hit, SilverLine was able to serve several hundred requests per second. We also note that latency (not throughput) is increasingly the factor that many Web applications face, so even this throughput reduction may be manageable in many cases.

Login Time vs. Number of Users. We measured how login redirection affects the number of concurrent login requests that the Web server can serve. We issued 5,000 login requests. For each trial, we increase load in terms of the number of concurrent requests per sec-

ond. We stopped increasing the concurrent load when the average response time for requests exceeds two seconds. We measured the maximum concurrent load per second that can be served within two seconds as the login throughput of the system. On the native server, the Web application could support 349 login requests per second. After enabling SilverLine, the application was able to support 280 requests, an effective throughput overhead of about 20%.

Response Time vs. Number of Users. Usually on e-commerce sites such as those where SilverLine might be deployed, different users are in different modes of interaction: some users are logging in while others are performing a transaction. To simulate this environment, we wrote a script that simulates a complete user session with logging in, adding a random item to a shopping cart, checking out, and logging out. These four actions typically generate more than ten HTTP requests. To see the performance of the site under such load, we ran the script for 5,000 users, increasing the number of concurrent sessions per second and observing performance. We stopped increasing the concurrent load when the average response time for each session exceeds 20 seconds. We measured the maximum concurrent load per second that can be served within 20 seconds as the user session throughput of the system. On the native server, we see an average 187 requests served per second; with SilverLine enabled, we measure 134 requests per second, an effective throughput cost of about 29%. We also measured how many concurrent users can be logged in simultaneously, and the effect of simultaneous user requests on latency. In this case, SilverLine could support approximately 987 concurrent users, and the 95th percentile of these requests were served in less than two seconds.

6. RELATED WORK

We present related work in data isolation, information-flow control, and language-level taint tracking. Table 3 summarizes the features of many related systems.

Data Isolation. CLAMP [27] and Nemesis [6] isolate data flows from different clients. CLAMP isolates code and data for each user using separate virtual machines, each of which is given only a limited view of the database using a *query restrictor*. Thus, even if the Web server or the Web application is compromised, an attacker can only retrieve data from the corresponding view. Due to the overhead of per-user virtual machines, CLAMP incurs higher overhead than SilverLine. CLAMP's query restrictor also restricts the Web application from performing aggregate queries (e.g., COUNT(*), AVG). In addition, the query restrictor does not allow queries on sensitive tables if the user is not logged in. Web applications sometimes provide statistics, even for anonymous users, in which case an architecture like CLAMP would require Web application code inspection and modification. One such example is the "create users" command in OSCommerce. For even a moderately complex application like HotCRP [17], CLAMP's data access control policies become complicated and would require review from domain experts. CLAMP also requires creating a virtual machine for every user session, which creates scaling problems for large Web server applications. Nemesis separates security code from the application code by having a different user authentication and tracking system than the application code itself, and authenticates the user twice, once at the application itself and then to Nemesis [6]. Nemesis places the Web application code in the trusted computing base, so it requires rewriting Web application code. CryptDB [29] protects user data by encrypting it and executing queries over an encrypted database to prevent unauthorized users from accessing data but does not have any taint-tracking capabilities.

Information-flow Control. Information flow control has been ap-

plied to systems since the 1970s [8]. Many operating systems have secured a *single* host against exploits or prevented data exfiltration, starting as early as 1975 with the Hydra operating system [5] to more recent work (e.g., Taos [35]). *Decentralized* information flow control [20] was initially designed as a new programming language paradigm, and specialized operating systems such as Asbestos [34] and HiStar [39] have adopted this approach to allow OS-level information-flow control between trusted and untrusted processes. Dstar [40] extends HiStar's decentralized information flow control to the network, using signed messages to transfer labels and capabilities between hosts. Unfortunately, Dstar requires HiStar on both the client and the server to facilitate label transfers—installing a new operating system is often not practical for Web servers, and for our purposes it is unrealistic to require Web clients to run a particular operating system. Flume [18] modifies Linux to allow similar policies as SilverLine, but it still requires applications trusted and untrusted applications be aware of information-flow control. In contrast, SilverLine allows applications to operate without explicit knowledge of information-flow policies (hence avoiding expensive application rewrites).

Language-level Taint Tracking. RESIN [38], Guardrails [3], PHPAspis [26] and DBTaint [7] use information-flow tracking at the programming language level to prevent information leaks. To prevent XSS and even injection, RESIN [38] allows programmers to check and sanitize their code: a programmer identifies the data-flow boundary and defines policy checks to be performed whenever data crosses that boundary. GuardRails [3] transforms annotated Ruby source code into secure code by generating similar checks automatically. Unfortunately, these systems require application code to be rewritten, and bring the core Web application code into the trusted computing base. PHPAspis improves on this trust model by only trusting parts of the code and performing taint tracking for only untrusted third-party plug-ins; because it does not monitor all application code, it can only defend against a limited range of attacks. DBTaint provides a cross-application fine-grained taint tracking platform by tainting database records, which is similar to SilverLine's approach of adding taints to database rows. However, DBTaint does not define specific security checks, and it also requires a modified language runtime.

Full-system Taint Tracking. Static "tainting" approaches [12, 20, 28, 33] can tag variables or portions of memory with security labels or bits, but these methods typically require rewriting applications to be aware of information-flow control. Dynamic taint analysis techniques such as TaintTracker [22], a mechanism to monitor information flows at the instruction level to detect potential exploits on a host; and process coloring [15], which tracks interactions between resources ("color diffusion") used for early detection of resources on a host that possess "colors" of a vulnerable process. Other efforts include Panorama [37], Privacy Oracle [16], TaintDroid [10], and Neon [41]. Although these systems can track data leaks from legacy applications, they (1) track information flow using a fixed set of non-expressive taints, (2) focus on data leaks from a single host, and (3) apply static policies only (e.g., tainting all data from external sensors). These systems also require instruction-level instrumentation, which introduces more overhead than SilverLine.

7. LIMITATIONS AND FUTURE WORK

Misconfiguration. In Section 4.2, we discussed how a new Web application can be configured to use SilverLine. From the database schemas that we inspected of the target category of applications, we found that finding the required configuration information is reasonably intuitive. Nevertheless, in case of a misconfiguration such

	RESIN	Guardrails	DBTaint	Flume	Nemesis	CLAMP	SilverLine
Can deploy across frameworks and languages	Requires porting	Only Ruby on Rails	Specific RDBMS	Yes	Specific framework	Yes	Yes section 4.1.3
Can deploy w/o changing application code	Yes	Requires Annotations	Requires DB schema change	Application needs to call IFC syscalls	Yes	Needs to change login and other modules of app	Needs to change login module 4.1.1
Performance	Limited due to instruction level tracking	No data available	Limited due to instruction level tracking	Reasonable overhead since process level taint tracking	Limited due to instruction level tracking	Limited due to slow VM spawning rate	Reasonable overhead due since process level taint tracking section 4 & 5
Data leakage prevention	✓	✓	×	✓	✓	✓	✓
Protection from XSS attacks	✓	✓	×	Limited	Limited	✓	×
Modularity	✓	×	×	×	✓	×	✓

Table 3: Comparison of related web security systems.

as a table with sensitive information being misclassified into a into a non-sensitive group, data leaks can still occur (although the leaks would be limited to data in that table). We believe that misconfiguration could be detected early, with test cases where a particular user’s session leaks information about another user. Because our taint storage granularity applies to an entire row of each table, a single test case per table should sufficient to detect misconfiguration. With such a sanity test mechanism in place, the administrator can iteratively refine the configuration of SilverLine.

False positives and false negatives. Any taint tracking system would be required to avoid (1) taint explosion, where information gets associated with way more taints than the correct number, resulting in false positives; and (2) a failure to associate the required taint with the sensitive information, resulting in false negative. False positives hampers the otherwise correct application functionality and is a trouble for administrators, while false negatives are silent and defeat the data-leak prevention. Because we are focusing on Web applications that have strong isolation between users’ data and sessions, we do not expect a user session to be tainted with more than one taint in normal operation. Still, some database queries can run across an entire sensitive table and aggregate results, which makes SilverLine most suitable for applications where a single user is accessing data that is not aggregated across multiple users (*e.g.*, banking, e-commerce). An application that runs aggregate queries with a WHERE clause are currently not supported. Such applications would require functionality change. We hope that we will identify such queries quickly as they break that part of application due to exfiltration prevention. Some other techniques that we have not investigated in details could be applying differential privacy measurements to such queries and letting the resulting answer pass through only if it is below a threshold.

In the absence of covert channels, false negatives are mainly a combination failure to identify sensitive information and failure to track the sensitive information. The failure to identify sensitive information can be fixed with a few test cases. Proving that taints are tracked correctly is more difficult job. In SilverLine, we track taints at the process level and we only do so at specific points which are system calls. Ultimately, it may be possible to create a formal model for information flow for each system call and prove the accuracy of taint tracking.

Protection against data modification. We focus on preventing information leaks. We believe that protection against acts where somebody deletes a row or drops a table can be mitigated with appropriate backups. Even then, of course, an attacker could modify

another user’s credentials and impersonate his identity. Although we do not address this problem, we believe that SilverLine could be modified to defend against such an attack by monitoring INSERT queries and verifying that a user session has the taint corresponding to the row that it is trying to modify. We describe this defense in our discussion of retrieving sensitive data (Section 3.2).

Partial Deployment. Administrators may wish to deploy SilverLine for only a subset of users. Because SilverLine does not modify the original application or the original application’s database schema, it can support such a partial deployment. In that case, the administrator must configure the list of users for whom the system would be enabled. The login module can periodically check whether the user belongs to this list. All of the other mechanisms such as taint retrieval and association, taint tracking, and declassification would be enabled only if that check is successful.

Integration with social networking applications. Social networking applications are not suitable for SilverLine, since they mainly thrive on information diffusion. Many Web applications now include some aspect of social networking such as “Share what I just did on Facebook”. This type of integration can be fixed by re-classifying sensitive information tables or by maintaining shadow database tables which are non-sensitive. Certain information might be explicitly declassified before it is shared with public users.

Integration with existing SDN controllers.. The current implementation of the SilverLine declassifier is a standalone application that resides on the path between the client and server. In an OpenFlow (or SDN)-enabled network, however, such an application could be implemented as an off-path network control application running at an SDN controller such as Floodlight [11] or Pox [30]. Moving these functions into a general SDN control application could ultimately allow information-flow policies to be expressed in higher-level languages such as Pyretic [19].

8. CONCLUSION

We presented the design, implementation, and evaluation of SilverLine, a system that prevents data leaks that might otherwise result from Web applications. SilverLine represents a different point in design space for Web application security systems: Rather than focusing on application modifications or data access control, SilverLine prevents exfiltration of sensitive data, even if the Web application itself is compromised. SilverLine uses process-level taint tracking to isolate code executing on behalf of various users. By applying taint-level protection to the data, rather than to the ap-

plication or execution environment, SilverLine can achieve strong guarantees, while scaling better and incurring less performance overhead than existing state-of-the-art systems. It also simplifies runtime checking for data leaks, since it does not rely on specifying complex data-access policies. Our prototype of SilverLine indicates that augmenting a wide variety of Web applications with SilverLine requires only modifications to the login process, not to any other part of the Web application.

Acknowledgments

This work was supported by NSF Award CNS-0916732 and CNS-0964647. We thank Jad Naous for detailed feedback.

REFERENCES

- [1] Amazon Relational Database Service. <http://aws.amazon.com/rds/>.
- [2] Attrition.net. Absolute Sownage. http://attrition.org/security/rant/sony_aka_sownage.html, 2011.
- [3] J. Burket, P. Mutchler, M. Weaver, M. Zaveri, and D. Evans. Guardrails: A data-centric web application security framework. In *WebApps*, 2011.
- [4] V. Business. Verizon Data Breach Investigations Report. http://www.verizonbusiness.com/resources/reports/rp_data-breach-investigations-report-2011_en_xg.pdf, 2011.
- [5] E. S. Cohen and D. Jefferson. Protection in the hydra operating system. In *Symposium on Operating Systems Principles (SOSP)*, pages 141–160, 1975.
- [6] M. Dalton, N. Zeldovich, and C. Kozyrakis. Nemesis: Preventing authentication & access control vulnerabilities in web applications. In *18th Usenix Security Symposium*, 2009.
- [7] B. Davis and H. Chen. Dbtaint: Cross-application information flow tracking via databases. In *Proceedings of the USENIX Conference on Web Applications*, June 2010.
- [8] D. E. Denning. A lattice model of secure information flow. *Communications of the ACM*, 19(5):236–243, May 1976.
- [9] L. Dignan. Sony's data breach costs. <http://www.zdnet.com/blog/btl/sonys-data-breach-costs-likely-to-scream-higher/49161>, 2011.
- [10] W. Enck, P. Gilbert, B. gon Chun, L. P. Cox, J. Jung, P. McDaniel, and A. N. Sheth. Taintdroid: An information-flow tracking system for realtime privacy monitoring on smartphones. In *Symposium on Operating Systems Principles (SOSP)*, Oct. 2010.
- [11] Floodlight OpenFlow Controller. <http://floodlight.openflowhub.org/>.
- [12] B. Hicks, S. Rueda, T. Jaeger, and P. McDaniel. From Trusted to Secure: Building Applications that Enforce System Security. In *Proceedings of the USENIX Annual Technical Conference*, Santa Clara, CA, June 2007.
- [13] InformationWeek. Stanford Hospital Breach Exposes 20,000 ER Records. <http://www.informationweek.com/news/security/attacks/231601110>, 2011.
- [14] T. Inquirer. Citibank Hacked by altering URLs. <http://www.theinquirer.net/inquirer/news/2079431/citibank-hacked-altering-urls>, 2011.
- [15] X. Jiang, A. Walters, F. Buchholz, D. Xu, Y.-M. Wang, and E. H. Spafford. Provenance-Aware Tracing of Worm Break-in and Contaminations: A Process Coloring Approach. In *ICDCS*, June 2006.
- [16] J. Jung, A. Sheth, B. Greenstein, D. Wetherall, and T. K. Gabriel Maganis. Privacy Oracle: a System for Finding Application Leaks with Black Box Differential Testing. In *ACM Conference on Computer and Communications Security*, 2008.
- [17] E. Kohler. Hot crap! In *In Proceedings of WOWCS*, Apr. 2008.
- [18] M. Krohn, A. Yip, M. Brodsky, N. Cliffer, M. F. Kaashoek, E. Kohler, and R. Morris. Information flow control for standard os abstractions. In *Proc. 21st ACM Symposium on Operating Systems Principles (SOSP)*, Stevenson, WA, Oct. 2007.
- [19] C. Monsanto, J. Reich, N. Foster, J. Rexford, and D. Walker. Composing Software-Defined Networks. In *USENIX NSDI*, 2013.
- [20] A. C. Myers. Jflow: practical mostly-static information flow control. In *POPL '99: Proceedings of the 26th ACM SIGPLAN-SIGACT symposium on Principles of programming languages*, pages 228–241, New York, NY, USA, 1999. ACM.
- [21] MySQL Proxy. http://forge.mysql.com/wiki/MySQL_Proxy.
- [22] J. Newsome and D. X. Song. Dynamic Taint Analysis for Automatic Detection, Analysis, and Signature Generation of Exploits on Commodity Software. In *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, San Diego, CA, 2005.
- [23] OpenFlow Switch Consortium. <http://www.openflowswitch.org/>, 2008.
- [24] osCommerce: Open Source online shop e-commerce solution. <http://www.oscommerce.com/>.
- [25] OWASP. OWASP Top 10 Application Security Risks - 2010. https://www.owasp.org/index.php/Top_10_2010-Main, 2010.
- [26] I. Papagiannis, M. Migliavacca, and P. Pietzuch. Php aspis: Using partial taint tracking to protect against injection attacks. In *2nd USENIX Conference on Web Application Development (WebApps)*, June 2011.
- [27] B. Parno, J. M. McCune, D. Wendlandt, D. G. Andersen, and A. Perrig. CLAMP: Practical prevention of large-scale data leaks. In *Proc. IEEE Symposium on Security and Privacy*, Oakland, CA, May 2009.
- [28] P. Broadwell, M. Harren, N. Sastry. Scrash: A system for generating security crash information. In *Proc. 12th USENIX Security Symposium*, Washington, DC, Aug. 2003.
- [29] R. A. Popa, C. M. S. RediñAeld, N. Zeldovich, and H. Balakrishnan. Cryptdb: Protecting confidentiality with encrypted query processing. In *Proc. 23rd ACM Symposium on Operating Systems Principles (SOSP)*, Cascais, Portugal, Oct. 2011.
- [30] POX OpenFlow controller. <http://www.noxrepo.org/pox/about-pox/>.
- [31] Redis. <http://redis.io/>.
- [32] Redis Benchmarks. <https://code.google.com/p/redis/wiki/Benchmarks>.
- [33] U. Shankar, K. Talwar, J. Foster, D. Wagner. Detecting format string vulnerabilities with type qualifiers. In *Proc. 10th USENIX Security Symposium*, Washington, DC, Aug. 2001.
- [34] S. VanDeBogart, P. Efstathopoulos, E. Kohler, M. Krohn, C. Frey, D. Ziegler, F. Kaashoek, R. Morris, and D. Mazieres. Labels and event processes in the Asbestos operating system. *ACM Transactions on Computer Systems*, 25(4):1–43, Dec. 2007.
- [35] E. Wobber, M. Abadi, and M. Burrows. Authentication in the Taos operating system. *ACM Transactions on Computer Systems*, 12(1):3–32, 1994.
- [36] C. Wright, C. Cowan, S. Smalley, J. Morris, and G. Kroah-Hartman. Linux Security Modules: General Security Support for the Linux Kernel. In *Proc. 11th USENIX Security Symposium*, San Francisco, CA, Aug. 2002.
- [37] H. Yin, D. Song, M. Egele, C. Kruegel, and E. Kirda. Panorama: capturing system-wide information flow for malware detection and analysis. In *Proceedings of the ACM Conference on Computer and Communications Security (CCS)*, Alexandria, VA, Oct. 2007.
- [38] A. Yip, X. Wang, N. Zeldovich, and M. F. Kaashoek. Improving application security with data flow assertions. In *Proc. of the 22nd ACM Symposium on Operating System Principles*, Oct. 2009.
- [39] N. Zeldovich, S. Boyd-Wickizer, E. Kohler, and D. Mazieres. Making Information Flow Explicit in HiStar. In *Proc. 7th USENIX OSDI*, Seattle, WA, Nov. 2006.
- [40] N. Zeldovich, S. Boyd-Wickizer, and D. Mazieres. Securing Distributed Systems with Information Flow Control. In *Proc. 5th USENIX NSDI*, San Francisco, CA, Apr. 2008.
- [41] Q. Zhang, J. McCullough, J. Ma, N. Scheer, M. Vrable, A. C. Snoeren, G. M. Voelker, S. Savage, and A. Vahdat. Neon: System Support for Derived Data Management. In *ACM Conference on Virtual Execution Environments*, Mar. 2010.